

DistroNexus — AI-Collaborative Programming: A Full Development Retrospective

Project: DistroNexus (WSL Distribution Manager)

Duration: 2026-01-22 ~ 2026-02-15 (25 days, 16 active development days)

Core philosophy: AI handles nearly all execution from research to delivery; the human developer serves solely as decision-maker and reviewer

1. Project Overview and Key Metrics

Dimension	Data
Git Commits	207
File Changes	1,686 file changes, +134,965 / -40,042 lines
Source Code	~26,900 lines (C# / XAML / PowerShell)
Test Code	~3,065 lines (xUnit + Pester)
Documentation	100 Markdown files, ~20,384 lines
Archived Docs	45 (completed plans/reports automatically archived)
Website	Docusaurus site, ~2,965 lines, bilingual EN/zh-CN
Releases	v1.0.1 → v1.0.2 → v2.0.1 (including a full stack rewrite)
Template System	15 built-in templates, 5 categories, 16 install scripts
PowerShell Cmdlets	15 automation commands

2. Development Timeline: Zero to Delivery in 25 Days

Phase 1: Prototype & v1.0 (Jan 22–25, 4 days)

Day 1 (1/22): Initial scripts → project restructuring → UI design doc → Go project init

Day 2 (1/23): Go/Fyne GUI → install/uninstall logic → CI pipeline → v1.0.1 release

Day 3 (1/24): v1.0.2 requirements → instance management → package manager → logging → CI/CD packaging

Day 4 (1/25): v1.0.2 fixes → Docusaurus website → deployment

What AI did in this phase:

- Implemented the complete Go/Fyne desktop application (GUI, install logic, uninstall logic)
- Set up GitHub Actions CI/CD pipeline (build, package, release)
- Wrote bilingual README, release notes, and WeChat promotion articles

- Built the Docusaurus i18n website and configured automatic deployment

Phase 2: Architecture Rewrite v2.0 (Jan 27–31, 5 days)

Day 5 (1/27): v2.0 requirements/task planning → PowerShell module infrastructure → all 15 Cmdlets
→ .NET solution setup → Core layer → WPF infrastructure → full WPF implementation
Day 6 (1/28): Phase 2 WPF refinement → Phase 3-4 integration/packaging/QA
Day 7 (1/29): Feature comparison docs → install wizard framework → download manager → cache manager
→ real-time log display → update source manager → auto-save settings
Day 8 (1/30): PowerShell module configuration optimization
Day 9 (1/31): Test infrastructure → integration validation → PowerShell→WPF migration
→ legacy Go code removal → disk size display

What AI did in this phase:

- **Stack migration execution:** Rewrote the entire project from Go/Fyne to .NET 10/WPF (human decision, AI execution)
- **Architecture design & implementation:** Full MVVM + DI + async/await architecture
- **PowerShell module:** Built all 15 Cmdlets from scratch
- **Test infrastructure:** Dual test framework with xUnit + Moq (C#) and Pester (PowerShell)

Phase 3: Feature Deepening & Internationalization (Feb 1–8, 5 days)

Day 10-11 (2/1-2): Keep-alive instance management → synchronous settings refactor → package manager enhancements
Day 12-13 (2/7-8): Full i18n localization → real-time locale switching plan → Copilot instructions refinement
→ documentation restructuring → UI compilation fixes → distribution source probing

What AI did in this phase:

- Generated complete resource files (EN/zh-CN bilingual), three-layer localization across XAML/ViewModel/Core
- Established documentation management standards (kebab-case naming, archival strategy, living-doc update rules)
- Analyzed and executed the SettingsService async-to-sync refactor (5KB file reads in <10ms, eliminating deadlock risks)
- Optimized startup performance: window display reduced from 3–5s to ~100ms ("show UI first, load data later")

Phase 4: Template System & Release (Feb 13–15, 3 days)

Day 14 (2/13): Template service core → desktop template UI → unit tests → requirements docs
Day 15 (2/14): Template execution engine → wizard step splitting → integration tests → 15 built-in template expansion
→ test stabilization → automation test suite → release preparation
Day 16 (2/15): Website documentation suite → template system release notes → packaging → v2.0.1 merge & release
→ zh-CN translation → WeChat promotion update

What AI did in this phase:

- Completed full-stack template system implementation: metadata → core service → UI integration → PowerShell commands
- Wrote install scripts for all 15 built-in templates (covering .NET/Node.js/Python/Docker/K8s/databases/AI-ML etc.)
- Built the automation test suite: Pass/Fail/Blocked classification, DryRun mode, NUnit/JUnit reports
- Wrote v2.0.1 release notes (bilingual), updated the website

3. AI Role Analysis: Full-Lifecycle Coverage

3.1 Responsibility Matrix

Development Phase	AI Responsibility	Human Responsibility
Requirements Research	Analyze WSL management needs, evaluate tech options	Set project direction
Architecture Design	MVVM/DI architecture, startup flow optimization	Approve architecture
Requirements Docs	Write functional requirements, acceptance criteria, task breakdowns	Confirm scope
Code Implementation	All source code (C#/XAML/PowerShell/Shell)	Code review
Testing	Unit tests, integration tests, automation test suites	Acceptance testing
Documentation	100 Markdown documents, bilingual	Content review
CI/CD	GitHub Actions workflows, build scripts	Trigger releases
Website	Docusaurus site build and content	Domain/deploy config
Promotion	WeChat promotional articles	Channel publishing
Refactoring	Go→.NET full stack migration	Approve migration decision
Performance	Startup time 3–5s→100ms	Validate experience

Development Phase	AI Responsibility	Human Responsibility
Release	Packaging, release notes, CHANGELOG	Final approval

3.2 Conventional Commits Distribution

Type	Count	Notes
feat	56	Feature implementation (27%)
docs	55	Documentation (27%)
chore	17	Engineering maintenance
refactor	17	Code refactoring
fix	15	Bug fixes
ci	8	Continuous integration
test	6	Test engineering
build	3	Build configuration

Feature implementation and documentation are nearly equal — a signature trait of AI-collaborative programming's "documentation-driven development", where AI needs high-quality context to maintain cross-session consistency.

4. Key Patterns of AI-Collaborative Programming

4.1 Documentation-Driven Development

This is the project's most distinctive AI-collaborative pattern. AI produces documentation before writing code at every phase:

Requirements doc → Task breakdown → Code implementation → Test documentation → Acceptance checklist → Archive

The planning triad: The project mandates three documents before any complex task:

- task_plan.md — Task breakdown and status tracking
- findings.md — Research conclusions
- progress.md — Progress log

Document lifecycle management:

- Active documents in docs/architecture/ and docs/development/ are updated alongside code changes

- Completed planning documents are automatically archived to `docs/archive/{year}/{month}/{yyyymmdd}_{topic}/`
- 45 archived documents demonstrate rigorous document lifecycle management

4.2 The Copilot Instructions System (`.github/copilot-instructions.md`)

The project uses a structured instruction file to "train" AI to maintain consistent coding style and engineering standards:

- **Language:** All code/comments/internal docs must be in English
- **Naming conventions:** C# PascalCase, PowerShell Verb-Noun, Conventional Commits
- **Coding patterns:** MVVM, async/await, constructor injection DI
- **Error handling:** Catch specific exceptions first, log everything, user-friendly UI messages
- **Planning rules:** Tasks with 3+ steps must create planning documents first

This instruction file itself went through multiple iterations (Chinese → English, simple → structured), reflecting the evolution of the "alignment protocol" between human developer and AI.

4.3 Stack Rewrite: AI Executes, Human Decides

The v1 → v2 stack rewrite is an extreme case of AI-collaborative programming:

Dimension	v1.0	v2.0
Language	Go	C# (.NET 10)
UI Framework	Fyne (cross-platform)	WPF (Windows native)
Architecture	Monolithic scripts	MVVM + DI + layered
Backend	Standalone PS scripts	PowerShell module (15 Cmdlets)
Testing	None	xUnit + Moq + Pester
Build	Manual	CI/CD automated

The core skeleton was completed in a single day (Jan 27): PowerShell module → .NET solution → Core layer → WPF layer — demonstrating AI's remarkable efficiency in large-scale code generation.

4.4 Iterative Problem Solving

The git log reveals typical AI debugging patterns:

- CI pipeline debugging: 6 consecutive fix commits (1/24 22:43–23:52) fixing runner, path, and permission issues
- Startup freeze issue: Multiple archived documents trace the complete diagnosis-to-resolution process
 - `startup-freeze-fix-report` → `startup-fix-test-checklist` → `startup-skip-settings-solution`
 - Ultimately resolved through SettingsService synchronous refactoring
- Compilation error fixes: Cross-XAML and C# resource reference consistency fixes

4.5 AI-Designed Testing Methodology

AI didn't just write test code — it designed a complete testing methodology:

1. **Layered testing strategy:** Unit tests (Moq) → Integration tests (WPF-PS bridge) → End-to-end automation
 2. **Template automation suite:** Self-designed Pass/Fail/Blocked tri-state classification system
 3. **Test evidence archiving:** Test results persisted as XML + JSON manifest + Markdown summary
 4. **Safety guards:** Real WSL tests disabled by default in CI, requiring explicit override
-

5. Release Cadence

```
1/22 — Project start (first commit)
1/23 — v1.0.1 release (Go/Fyne first version, Day 2)
1/24 — v1.0.2 release (feature enhancement, Day 3)
1/25 — Website launch
1/27 — v2.0 rewrite begins (stack switch decision)
2/15 — v2.0.1 release (complete rewrite, Day 16)
      Total: 25 days, 3 version releases
```

6. Project Deliverables

6.1 Software Products

- **DistroNexus Desktop** — WPF desktop app (.NET 10, Fluent Design, dark mode)
- **DistroNexus PowerShell Module** — 15 automation Cmdlets
- **15 built-in WSL templates** — One-click development environment setup
- **Windows installers** — Inno Setup installer + portable ZIP + self-contained publish
- **Official website** — Docusaurus bilingual site

6.2 Engineering Assets

- 207 structured Git commits (following Conventional Commits)
- 100 documentation files (requirements/design/guides/release notes/promotion)
- CI/CD pipeline (GitHub Actions, automated build & release)
- Complete test infrastructure (C# + PowerShell)
- Document archival system (45 archived planning documents)

6.3 Quality Assurance

- Startup performance KPIs: Window visible in <200ms, ~80–90MB memory
 - Security mechanisms: Path traversal detection, absolute path rejection, input sanitization
 - Multi-layer exception handling: Dispatcher + AppDomain + TaskScheduler
 - 15s timeout-protected async data loading
-

7. Key Insights and Takeaways

7.1 AI-Collaborative Programming Multiplies Efficiency

- **25 days** to complete what would traditionally take **months**
- Including a **complete tech stack rewrite** (Go → .NET)
- Source code + tests + docs + website totaling **50,000+ lines** of output
- Human effort focused on: **direction decisions, requirement confirmation, final acceptance**

7.2 Documentation Is AI's "Memory"

Documentation volume (20,384 lines) approaches source code volume (26,918 lines). This isn't over-documentation — it's an inherent requirement of AI-collaborative programming. Each new AI session needs high-quality context to quickly restore state and maintain consistency.

7.3 Copilot Instructions = AI's "Engineering Culture"

[.github/copilot-instructions.md](#) is the "alignment protocol" between human developer and AI. Its multiple iterations (simplify → add planning rules → documentation management norms) reflect how humans gradually optimize the collaboration interface with AI.

7.4 The Irreplaceability of the Human Decision-Maker

Despite AI completing nearly all execution work, the following key decisions were made by humans:

- **Project direction:** Targeting WSL management
- **Stack migration:** The strategic decision to move from Go → .NET 10/WPF
- **Release cadence:** When to release, what to release
- **Quality threshold:** Final acceptance standards
- **Promotion strategy:** WeChat channel selection

8. Acknowledgment

From the v2.0.1 release notes:

"AI completed nearly the entire end-to-end workflow of this project — from requirements research, development, and testing to release delivery, including this release notes document. With AI handling almost all execution work, I was finally able to invest time in the output I have wanted to create for a long time."

This document was generated by AI (Claude Opus 4.6) based on analysis of 207 Git commits and 100 project documents.

Date: 2026-02-15